

HARPia - Detecção de Pessoas em Vídeo com YOLO

Material técnico para estudo e apresentação da prova de conceito desenvolvida no repositório `Phantom-root-br/yolo-inference`.

A entrega valida percepção visual em isolamento. Não há controle PX4, arming, takeoff ou navegação neste repositório.

Resumo executivo

A tarefa foi transformar uma inferência YOLO simples em uma POC reproduzível de detecção de pessoas em vídeo no ambiente HARPia. O pipeline usa OpenCV para decodificar frames, YOLO11n para detectar exclusivamente a classe person, desenha bounding boxes, grava vídeo anotado, salva métricas estruturadas e preserva evidências visuais.

Indicador	Resultado
Modelo	YOLO11n pré-treinado
Hardware	Intel Core i3-3217U, CPU-only
Vídeo	1280x720, 29,970 FPS, 1137 frames
Parâmetros	imgsz=640, conf=0.25, device=cpu
Tempo total	449,202 s
FPS efetivo	2,531
Tempo médio/frame	395,077 ms
Inferência YOLO média	354,935 ms/frame
Detecções acumuladas	7044 boxes per son
Confiança média	0,5562

7044 não representa pessoas únicas. É a soma de bounding boxes ao longo dos frames; não existe tracking nesta POC.

Objetivo e escopo

O objetivo foi validar o fluxo completo de percepção sem alterar a stack de voo:

```
vídeo público
-> OpenCV
-> frames
-> YOLO11n
-> filtro person
-> bounding boxes + confiança
-> vídeo anotado + métricas + evidências
```

Dentro do escopo:

- inferência frame a frame;

- classe `person` encontrada programaticamente em `model.names`;
- bounding boxes e confiança;
- vídeo anotado preservando o FPS original;
- métricas JSON;
- frames representativos;
- benchmark YOLO11n x YOLO11s;
- documentação, logs, painel visual e CI.

Fora do escopo:

- treinamento ou fine-tuning;
- CUDA/GPU;
- tracking de identidade;
- avaliação com ground truth;
- ROS 2 persistente;
- controle PX4.

Ambiente validado

- Intel Core i3-3217U @ 1.80 GHz;
- 2 núcleos físicos / 4 threads;
- sem GPU NVIDIA;
- CUDA indisponível;
- Python 3.10.12;
- PyTorch CPU;
- Ultralytics 8.4.158;
- OpenCV 5.0.0;
- ambiente isolado `/root/yolo_venv`.

O PyTorch emitiu warnings de NNPACK devido ao processador antigo. O detector desabilita esse backend quando disponível. Isso não desliga a inferência em CPU e não tem relação com CUDA.

Fonte de vídeo

Foi utilizado um vídeo público Creative Commons com múltiplas pessoas:

- título: Blurred Crowd of People Walking - Free Stock Creative Commons Video;
- uploader: Freestocks;
- ID: 0EFv_pvFQyk;
- duração nominal: 38 s;
- resolução validada: 1280x720;
- FPS: 29,97002997;

- frames: 1137.

O vídeo bruto não é versionado. `scripts/download_video.sh` reproduz o download.

Metodologia

- 1 Baixar e validar o vídeo.
- 2 Abrir a mídia com OpenCV.
- 3 Carregar YOLO11n em CPU.
- 4 Descobrir o ID da classe person em `model.names`.
- 5 Processar cada frame com `imgsz=640` e `conf=0.25`.
- 6 Extrair bbox e confiança.
- 7 Desenhar boxes e labels.
- 8 Gravar o frame anotado no FPS original.
- 9 Acumular métricas de desempenho e detecção.
- 10 Salvar JSON e frames representativos.

Execução completa:

```
/root/yolo_venv/bin/python scripts/detect_people.py \  
--input input/people_cc.mp4 \  
--output output/yolo11n_people.mp4 \  
--model yolo11n.pt \  
--imgsz 640 \  
--conf 0.25 \  
--device cpu \  
--metrics results/yolo11n_metrics.json \  
--examples 3 \  
--examples-dir results/examples
```

Resultados completos - YOLO11n

Métrica	Valor
Frames processados	1137
Duração da saída	37,9379 s
Tempo de processamento	449,2024 s
Tempo médio por frame	395,0768 ms
FPS efetivo	2,5312
Inferência YOLO média	354,9351 ms/frame
Detecções person	7044
Confiança média	0,5562
Confiança mínima	0,2500
Confiança máxima	0,9093
Tamanho do peso	5,3537 MB

O processamento ficou aproximadamente 11,84 vezes abaixo do tempo real em relação aos 29,97 FPS da fonte. Isso mede capacidade de processamento, não velocidade de reprodução do arquivo.

FPS da fonte x FPS de processamento

- **FPS da fonte:** taxa de reprodução do vídeo original.
- **FPS de processamento:** quantos frames a máquina consegue inferir por segundo.
- **FPS do arquivo de saída:** o VideoWriter usa o FPS original.

Assim, a máquina pode levar vários minutos para produzir um vídeo de 38 s, mas o arquivo final ainda reproduz em aproximadamente 38 s.

Evidências visuais

A execução completa preservou:

- `results/examples/frame_000252.jpg;`
- `results/examples/frame_000269.jpg;`
- `results/examples/frame_000350.jpg.`

O critério prioriza frames com mais pessoas detectadas e, em empate, maior soma das confidências.

Benchmark YOLO11n x YOLO11s

Ambos os modelos foram executados nos mesmos 60 frames e com os mesmos parâmetros.

Métrica	YOLO11n	YOLO11s
Frames	60	60
Tempo total	40,736 s	57,589 s
FPS	1,473	1,042
ms/frame	678,925	959,812
YOLO ms/frame	568,863	862,467
Deteções	425	422
Confiança média	0,5533	0,5887
Tamanho	5,35 MB	18,42 MB

YOLO11s consumiu cerca de 1,41x mais tempo por frame e é 3,44x maior. A confiança média foi um pouco maior, mas o total de detecções não aumentou. Por isso, YOLO11n foi mantido como baseline.

Observabilidade e logs

A camada operacional não altera o detector. Ela organiza execução, logs e demonstração:

- `run_video_poc.sh`: modos `test`, `full` e `report`;

- `watch_video_log.sh`: `tail -f` do log mais recente;
- `video_report.py`: painel HTML com vídeo, métricas, evidências e log;
- `transcode_web_video.py`: H.264/yuv420p para reprodução no navegador;
- `harpia-copylog`: cópia do log quando disponível.

Modo de apresentação sem refazer inferência:

```
./scripts/run_video_poc.sh report  
/root/yolo_venv/bin/python -m http.server 8000 --bind 0.0.0.0
```

Acesse `http://localhost:8000/output/video_report.html`.

Compatibilidade H.264

O OpenCV grava o vídeo anotado em mp4v. Navegadores podem não reproduzir esse codec. A solução foi transcodificar o MP4 já anotado para:

- H.264 / `libx264`;
- yuv420p;
- `+faststart`.

As bounding boxes já estão incorporadas aos pixels do frame antes da transcodificação, então a conversão preserva as detecções e não executa YOLO novamente.

Qualidade de software

A entrega foi organizada por branches e pull requests, com:

- Ruff;
- Pytest;
- GitHub Actions;
- squash merge;
- pesos, vídeos e logs grandes fora do Git;
- métricas e evidências pequenas versionadas.

Limitações

- CPU antiga e sem GPU;
- inferência offline;
- sem tracking;
- sem ground truth para precision/recall/mAP;
- pesos COCO genéricos;
- benchmark prático, não científico abrangente;
- sem controle PX4 nesta etapa.

Próximos passos

- 1 Trocar o vídeo offline pela câmera do HARPIa.
- 2 Medir latência ponta a ponta.
- 3 Publicar detecções e imagem de debug.
- 4 Encapsular a percepção em ROS 2 somente depois da validação da entrada contínua.
- 5 Construir dataset e ground truth específicos.
- 6 Medir precision/recall/mAP.
- 7 Avaliar fine-tuning.
- 8 Só depois conectar seleção de alvo e controle.

Roteiro de apresentação

- Problema: validar percepção sem mexer na stack de voo.
- Arquitetura: vídeo -> OpenCV -> YOLO -> person -> boxes -> métricas.
- Resultado: 1137 frames, 7044 boxes, 2,531 FPS.
- Benchmark: 11n mais adequado que 11s no hardware atual.
- Observabilidade: logs, painel e vídeo H.264.
- Limitações e próximos passos.

A entrega não foi apenas fazer o YOLO rodar. Foi transformar uma inferência isolada em uma POC reproduzível, mensurável e compartilhável, com decisão de modelo baseada em dados.